

데이터의 노이즈를 걸러내는 기술

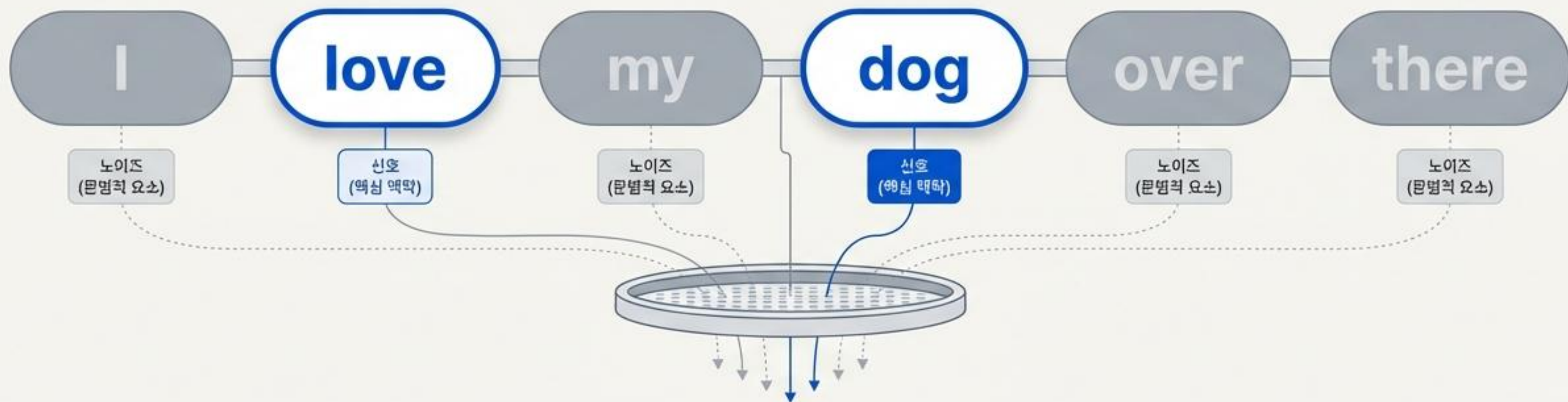
자연어 처리(NLP)를 위한 불용어(Stopwords) 완벽 가이드

- NLTK를 활용한 영어 표준 처리
- KoNLPy 기반 한국어 맞춤형 처리
- 효율적인 텍스트 정제 파이프라인 구축



불용어(Stopwords)의 정의

문장 내 등장 빈도는 높지만, 실제 '의미 분석'에는 기여하지 않는 단어들



영어의 대표적 불용어

'I', 'my', 'me', 'over', 'the' 등 대명사와 전치사

한국어의 대표적 불용어

'은', '는', '이', '가', '을', '를' 등 각종 조사 및 접미사

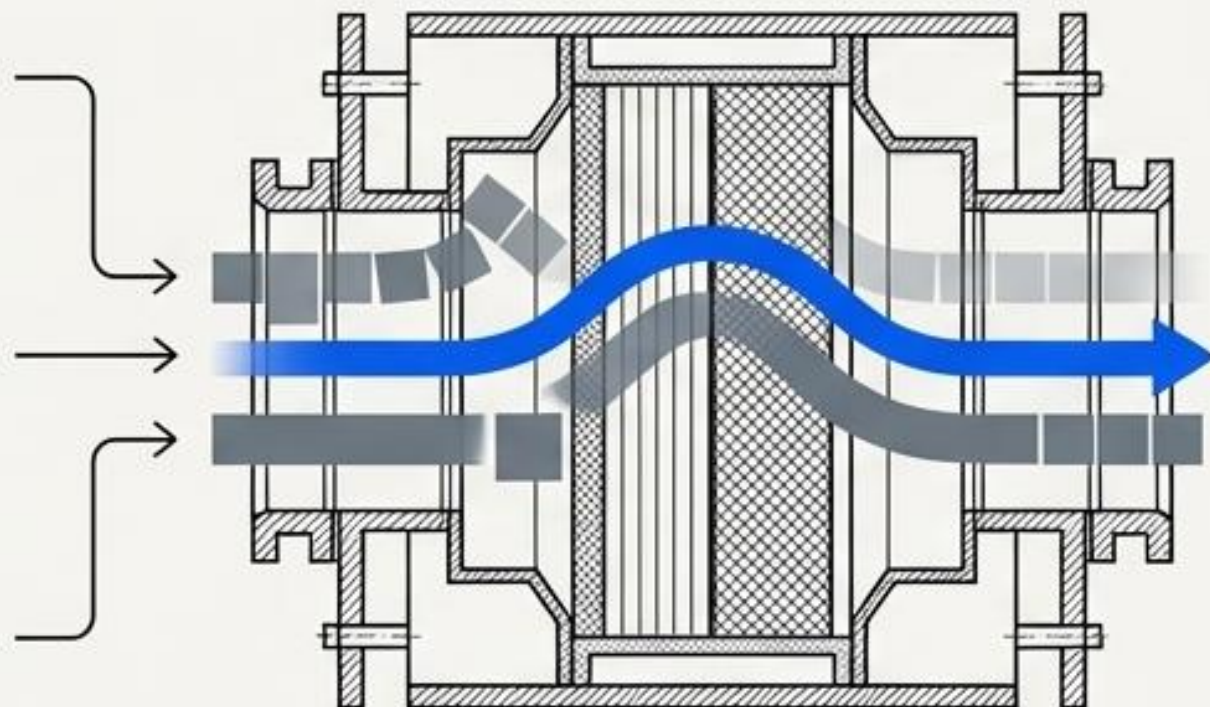
왜 불용어를 걸러내야 하는가?

원시 데이터 (Raw Data)



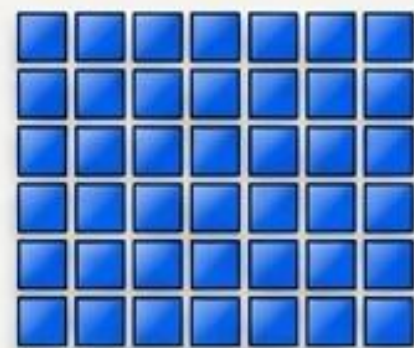
수많은 불필요한
토큰 혼재

불용어 필터링 (Stopword Removal)



불용어 필터링
(Stopword Removal)

정제된 데이터 (Clean Data)



차원 축소 및
핵심 가중치 집중



데이터 차원 축소

불필요한 단어 토큰을 제거하여
전체 데이터 크기 대폭 감소



연산 효율성 극대화

머신러닝/딥러닝 모델의
훈련 시간 단축 및 메모리 절약



분석 정확도 향상

문서의 주제를 결정하는 유의미한
단어에 모델의 가중치 집중

실습 환경 준비 : 두 가지 핵심 라이브러리

영어 텍스트 처리

NLTK (Natural Language Toolkit)

- 내장된 100여 개 이상의 기본 불용어 사전 사용 (Standardized)

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
```

한국어 텍스트 처리

KoNLPy & Okt 분석기

- 형태소 분석 후 사용자 직접 정의 방식 채택 (Customized)

```
from konlpy.tag import Okt
okt = Okt()
```

두 도구 모두 파이썬(Python) 환경에서 완벽하게 통합 구동

영어 실습 1단계: NLTK 데이터 로드

1

문제점 (The Problem)

로컬 환경 (Local Environment)

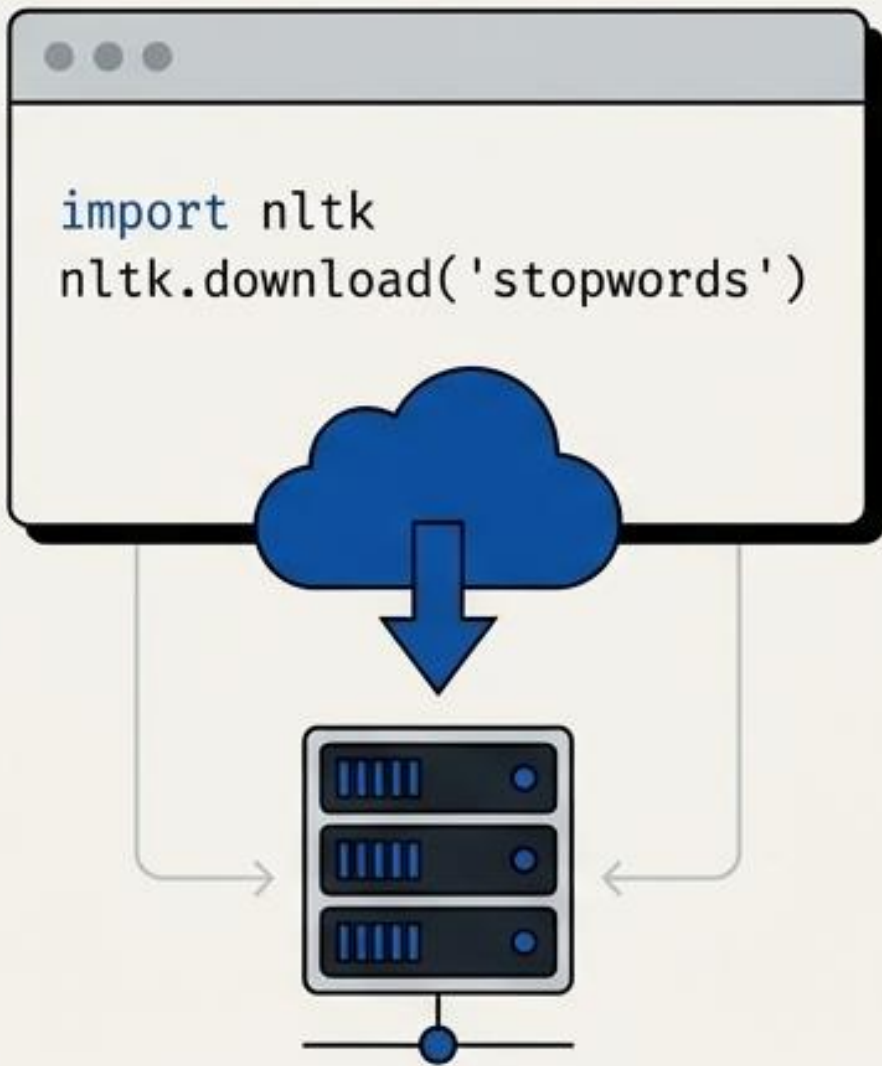


! Data Not Found Error

NLTK 패키지 내 기본 불용어 리스트를 사용하기 위해 사전 데이터가 필수적으로 요구됨.

2

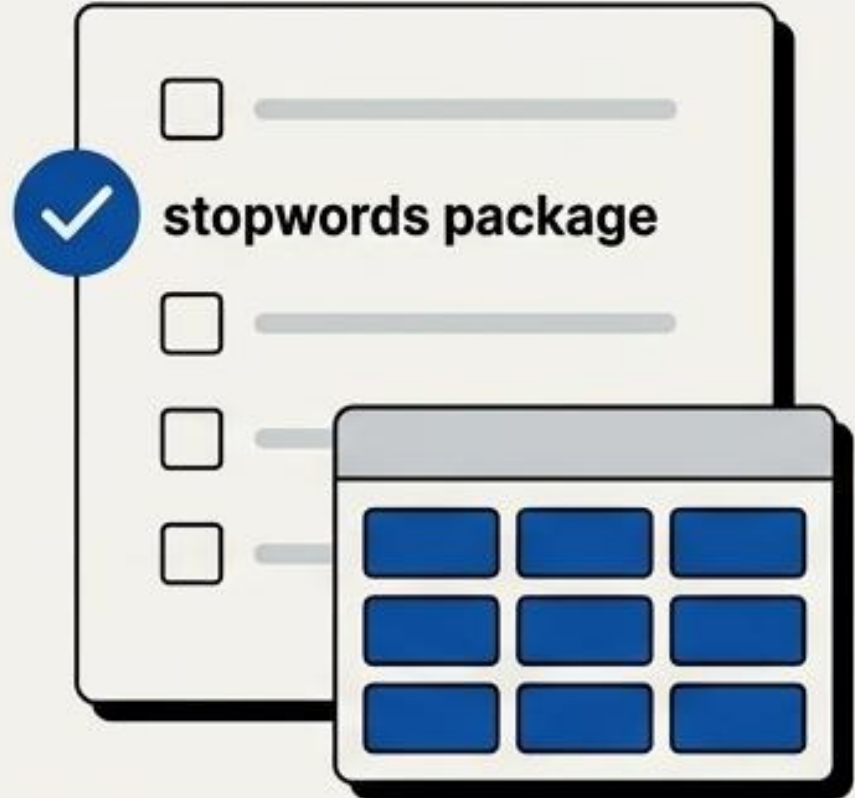
실행 (The Execution)



```
import nltk
nltk.download('stopwords')
```

3

해결 (The Resolution)



stopwords package

다운로드 완료.
100여 개 이상의 영어 불용어 세트가 로컬 메모리에 적재됨.

NLTK 영어 불용어 사전 확인: 전체 규모 측정

```
stop_words_list = stopwords.words('english')  
print('불용어 개수 :', len(stop_words_list))
```

179

사전 정의된 영어 불용어 총 개수

NLTK의 stopwords.words('english') 메서드는 영어 분석에 있어 가장 보편적으로 제외되어야 할 179개의 필수 토큰 배열을 즉시 반환합니다.

NLTK 영어 불용어 사전 확인: 내부 데이터 딥다이브

```
print('불용어 10개 출력 :', stop_words_list[:10])
```

슬라이싱 문법을 통한
샘플 추출

‘i’

‘me’

‘my’

‘myself’

‘we’

‘our’

‘ours’

‘ourselves’

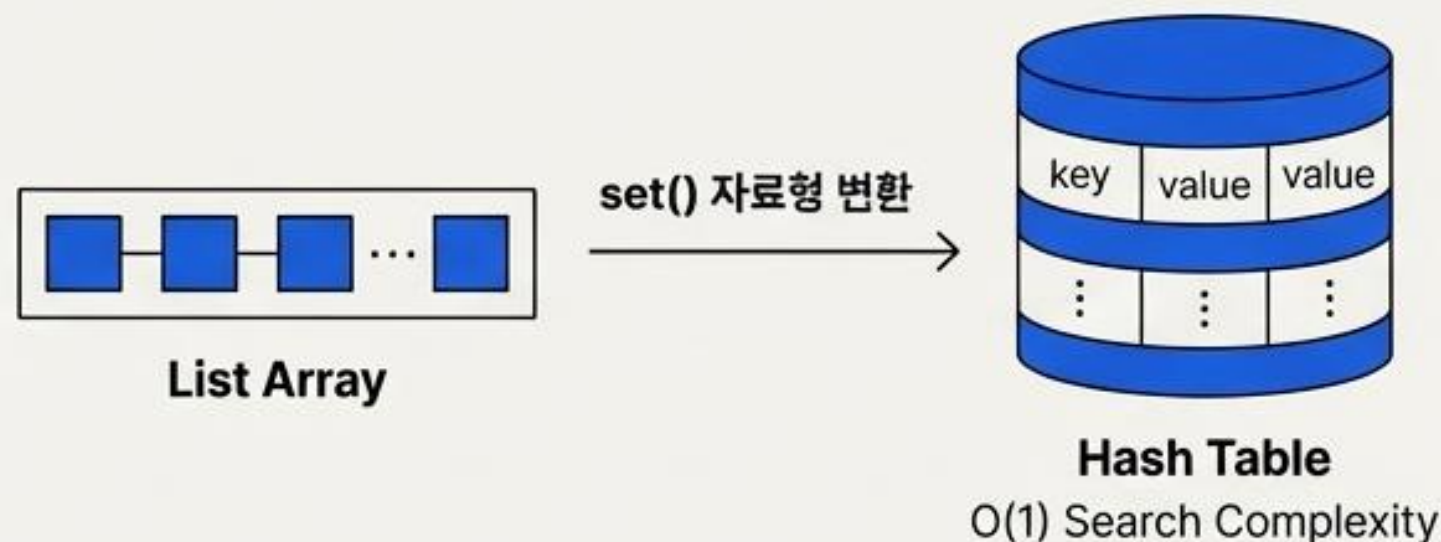
‘you’

“you're”

데이터 특징 분석: 1인칭 및 2인칭 대명사, 소유격, be동사의 축약형 등 의미 분석에 불필요한 문법적 뼈대 단어들로 구성되어 있음을 확인.

“Family is not an important thing. It's everything.”

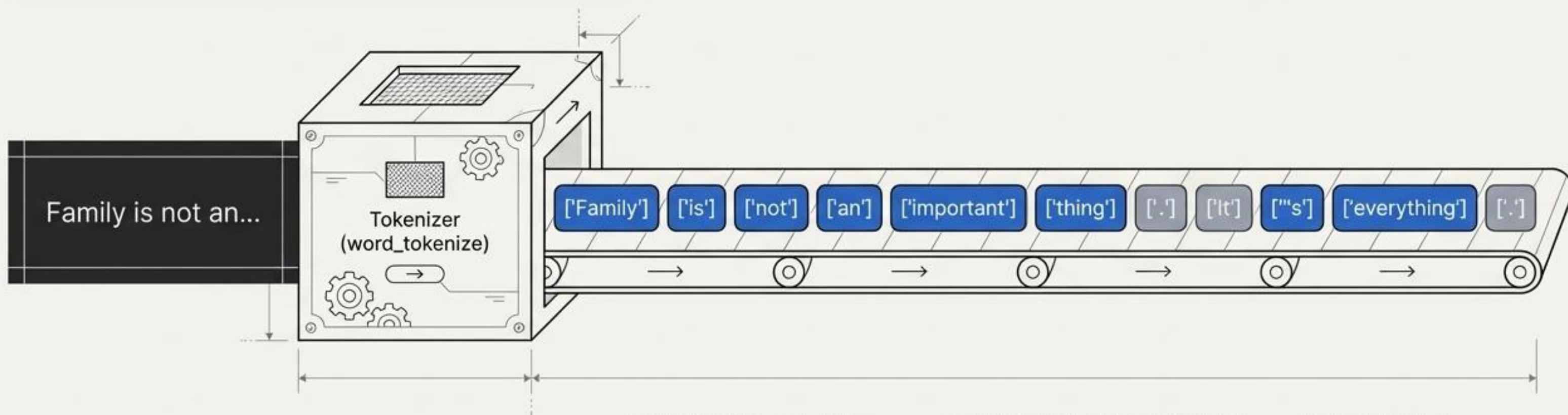
```
example = "Family is not an important thing. It's everything."  
  
stop_words = set(stopwords.words('english'))
```



이후 진행될 반복문(for loop)에서의 탐색 속도와 효율성을 극대화하기 위해 리스트를 중복 없는 집합(Set) 형태로 캐스팅.

영어 실습 3단계: 문장 토큰화(Tokenization)

```
word_tokens = word_tokenize(example)
```

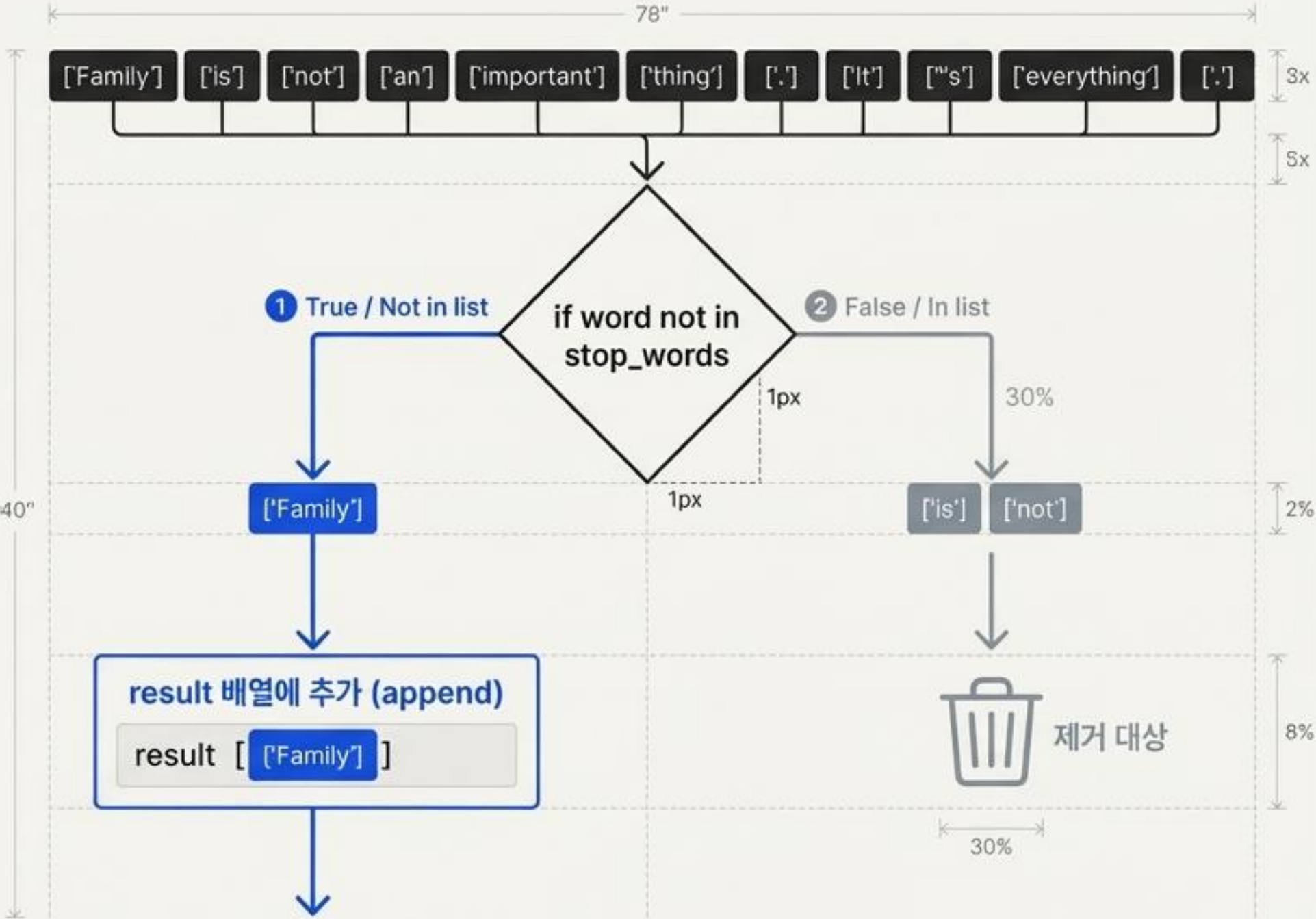


문장 단위의 스트링(String)을 개별 단어 단위의 배열(Array)로 분리 완료.
현 단계에서는 불용어와 유의미한 단어가 혼재되어 있는 원시 상태.

영어 실습 4단계: 논리적 필터링 (The Checkpoint)

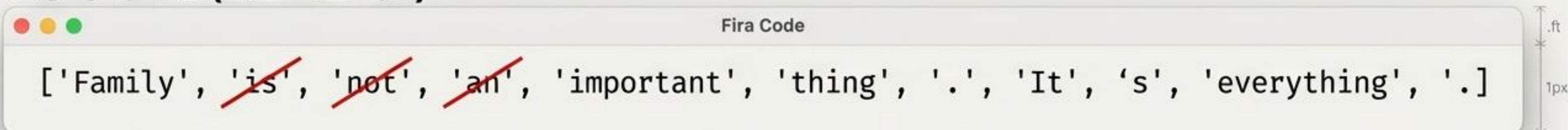
Fira Code

```
result = []  
  
for word in word_tokens:  
    if word not in stop_words:  
        result.append(word)
```



영어 실습 결과: Before vs. After 비교

불용어 제거 전 (원본 토큰 배열)



```
['Family', 'is', 'not', 'an', 'important', 'thing', '.', 'It', 's', 'everything', '.']
```

The image shows a Fira Code code editor window. The text inside is a list of tokens: ['Family', 'is', 'not', 'an', 'important', 'thing', '.', 'It', 's', 'everything', '.']. The words 'is', 'not', and 'an' are crossed out with red diagonal lines. The window has a title bar with 'Fira Code' and three colored window control buttons (red, yellow, green) on the left. On the right side of the image, there are vertical dimension lines: one labeled '.ft' and another labeled '1px'.

불용어 제거 후 (정제된 배열)



```
['Family', 'important', 'thing', '.', 'It', 's', 'everything', '.']
```

The image shows a Fira Code code editor window. The text inside is a list of tokens: ['Family', 'important', 'thing', '.', 'It', 's', 'everything', '.']. Each token is enclosed in a blue rectangular box. The window has a title bar with 'Fira Code' and three colored window control buttons (red, yellow, green) on the left. On the right side of the image, there are vertical dimension lines: one labeled '.ft' and another labeled '40"'. At the bottom of the image, there is a horizontal dimension line labeled '78"'. The entire diagram is enclosed in a light gray border.

- 총 3개의 불용어('is', 'not', 'an')가 성공적으로 식별 및 제거됨
- 마침표 ('.') 및 어포스트로피('s')와 같은 구두점은 사전에 없으므로 보존됨
- 문장의 뼈대를 이루는 핵심 명사와 형용사 중심의 정제 완료

한국어 불용어 처리의 특수성: 왜 더 복잡한가?

영어: 고립어 (띄어쓰기 중심의 독립 구조)

[I]

[love]

[you]

단순한 단어 매칭만으로도 필터링 가능

한국어: 교착어 (형태소가 결합된 복합 구조)

[나]

[는]

[너]

[를]

[사랑해]

단순 매칭 불가. 형태소 분석(Morphological Analysis)을 통한 정밀 분리 선행 필수

한국어 불용어 사전: 사용자 정의(Customization)의 필수성



분석하려는 텍스트의 도메인 문맥에 따라 명사나 형용사 중에서도 노이즈로 취급해야 할 단어가 수시로 발생. 개발자가 직접 사전을 구축하고 지속 업데이트해야 함.

한국어 실습 1단계: 타겟 문장 및 맞춤형 문자열 정의

“

고기를 아무렇게나 구우려고 하면 안 돼. 고기라고 다 같은 게 아니거든. 예컨대 **삼겹살**을 구울 때는 **중요한** 게 있지.

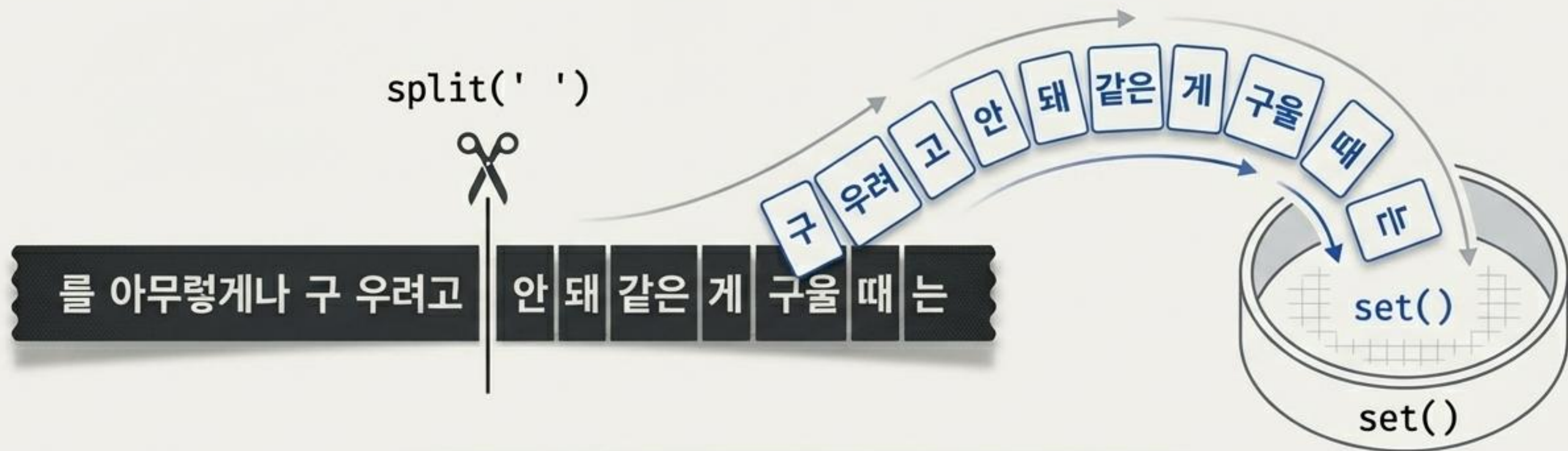
”



```
stop_words = "를 아무렇게나 구 우려고 안 돼 같은 게 구울 때 는"
```

→ 해당 문맥에서 의미적 가치가 없다고 판단되는
형태소들을 하나의 긴 문자열(String)로 정의

한국어 실습 2단계: 파싱 및 Set 자료형 변환



```
stop_words = set(stop_words.split(' '))
```

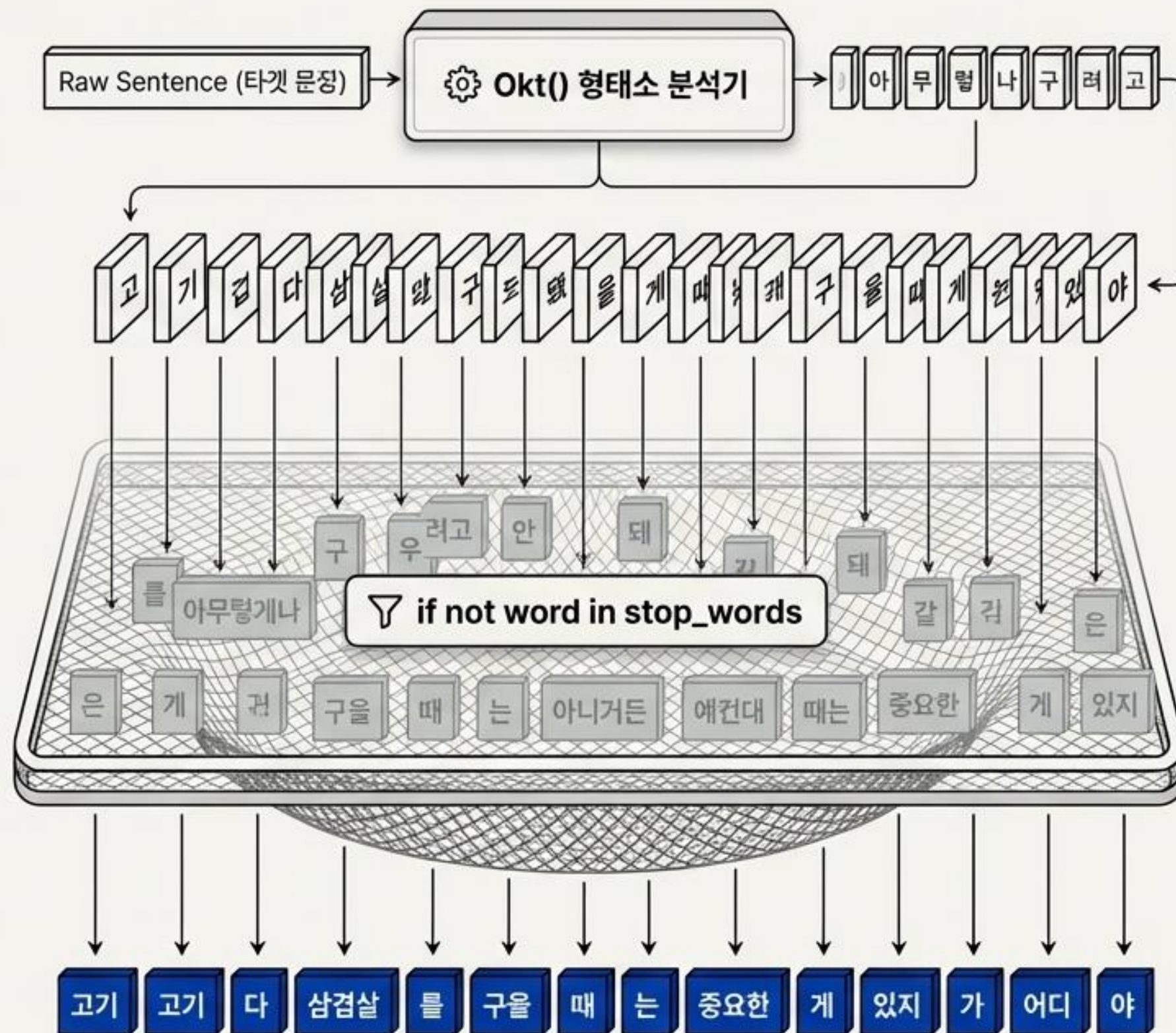
- 띄어쓰기를 기준으로 단일 문자열을 리스트 배열로 분절
- 중복 검사 성능 최적화를 위해 집합(Set) 구조로 최종 캐스팅 완료

한국어 실습 3단계: 형태소 분석 및 컴프리헨션 필터링

```
word_tokens = okt.morphs(example)

result = [word for word in word_tokens
          if not word in stop_words]
```

리스트 컴프리헨션(List Comprehension)을 활용한 1줄 처리



한국어 실습 결과:

형태소 단위 Before vs. After 비교

불용어 제거 전 (총 27개 토큰)

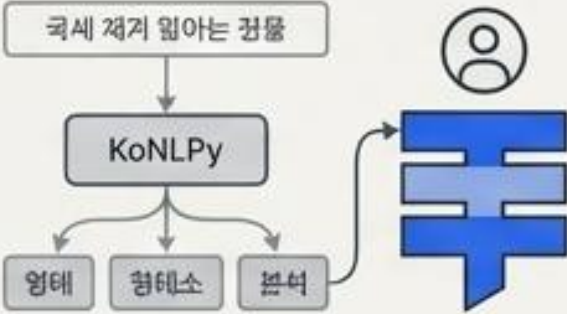
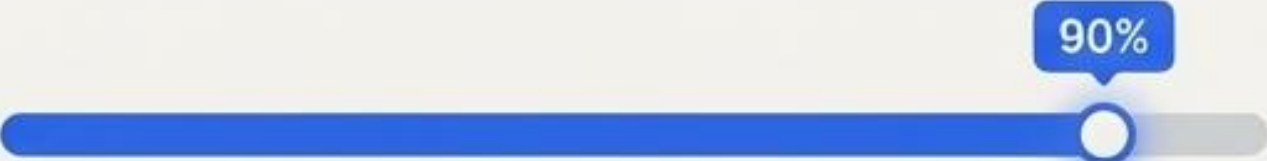
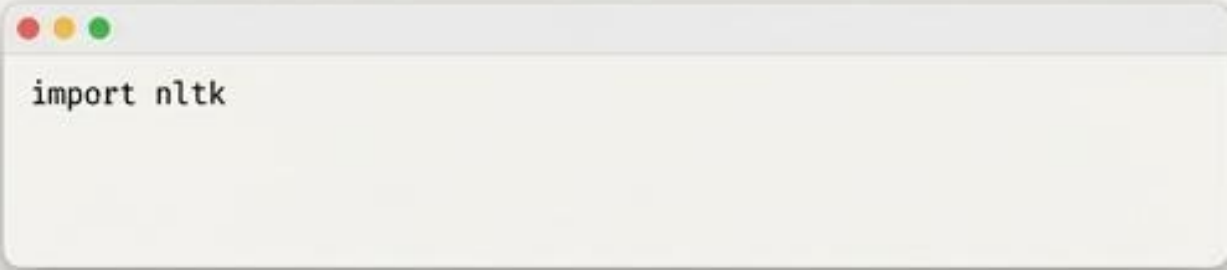
['고기', '~~'를'~~', '~~'아무렇게나'~~', '~~'구'~~', '~~'우려'~~', '~~'고'~~', '~~'한다면'~~', '~~'안'~~' '~~'돼'~~', '.. '고기' '라고' '다'
'~~'같은'~~' '게' '아니거든' .. 예컨대 '삼겹살' '을', '~~'구을'~~' '~~'때'~~' '~~'는'~~', '중요한' '게' '있지', '.']

불용어 제거 후 (핵심 14개 토큰)

['고기', '하면', '.', '고기', '라고', '다', '아니거든', '.', '예컨대' '삼겹살' '을' '중요한' '있지' . ']

수많은 형태소(조사, 어미 등)로 쪼개졌던 원본 토큰 배열이
프로젝트 문맥을 전달하는 명사/형용사 위주로 완벽히 압축됨.

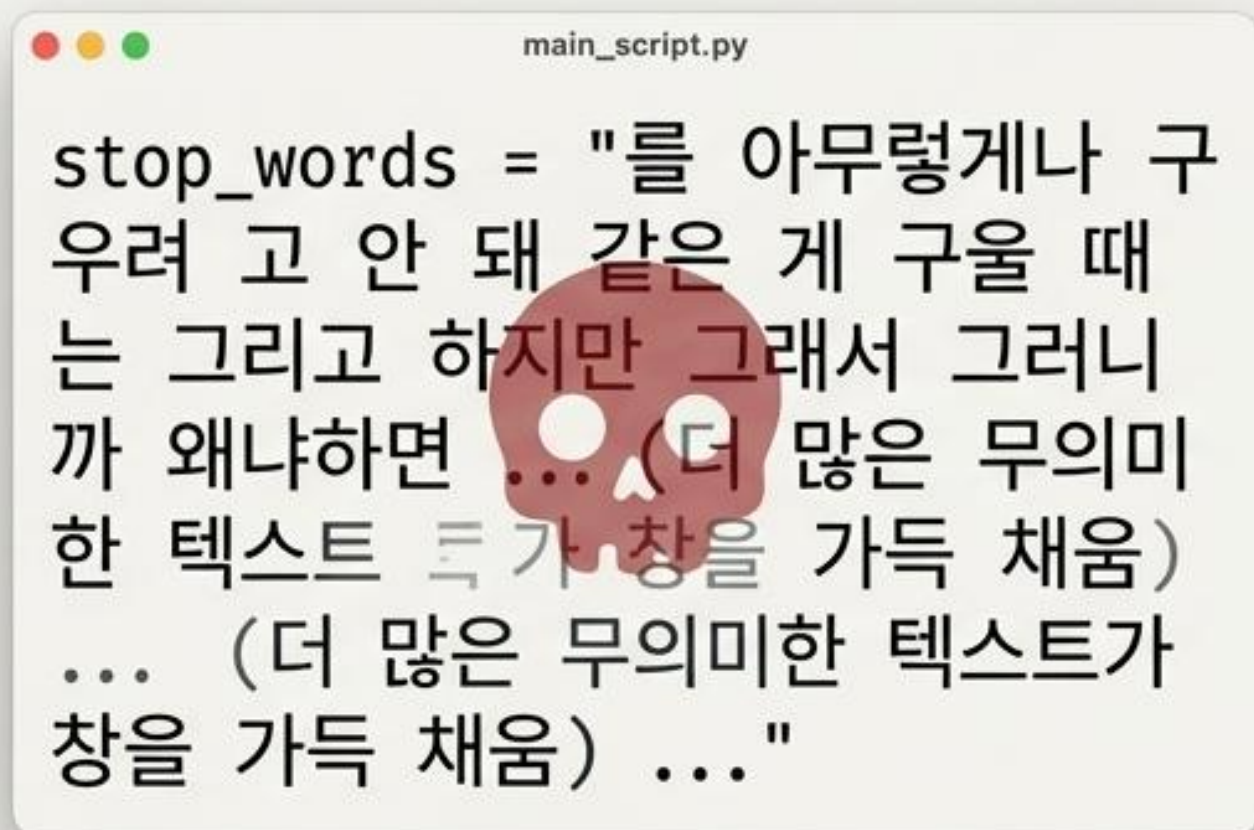
방법론 종합 비교: 표준화(Standard) vs 맞춤형(Custom)

	영어 / NLTK 기반 (Top-Down)	한국어 / KoNLPy 기반 (Bottom-Up)
사전 구성 방식	사전 검증된 179개 표준 사전 패키지 로드 후 즉시 사용 	형태소 분석 후 프로젝트 도메인에 맞춰 개발자가 직접 문자열 직조 
유연성 및 도메인 적합도	낮음 (Low) - 범용적이나 특수 도메인 대처 한계 	매우 높음 (High) - 모든 문맥에 대한 완벽한 통제권 확보 
구현 난이도	매우 쉬움 (단순 패키지 Import) 	다소 까다로움 (형태소 분리 및 커스텀 리스트 관리 필요) 

실무 모범 사례 (Best Practice)

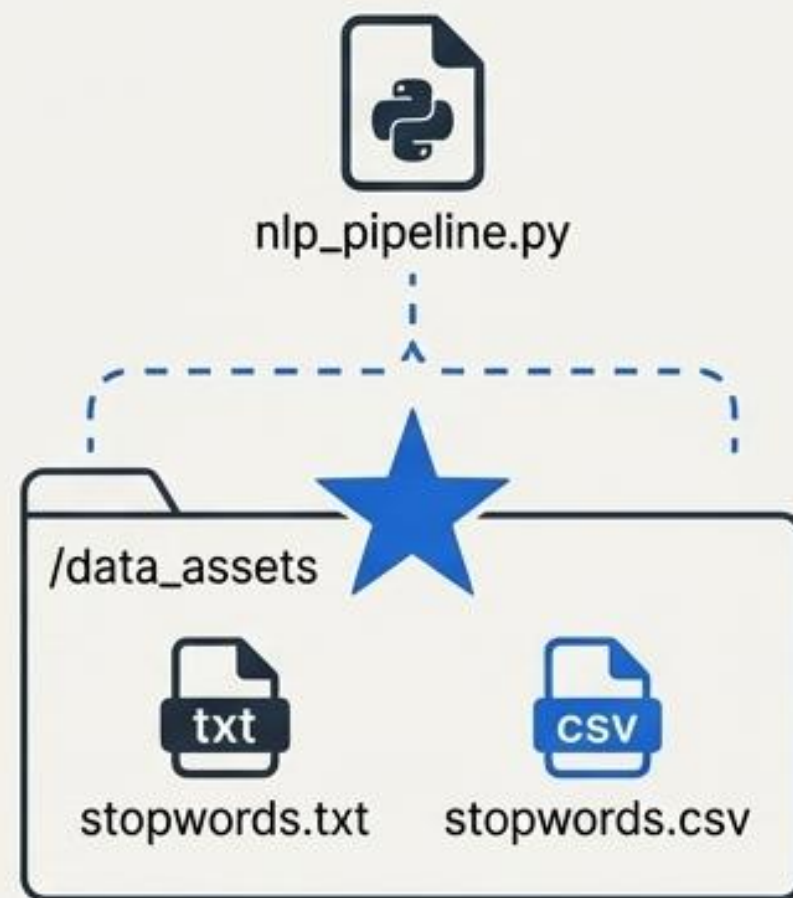
1: 대규모 사전 관리 구조

Bad Practice



파이썬 스크립트 내부 하드코딩 (유지보수 치명적 오류)

Best Practice

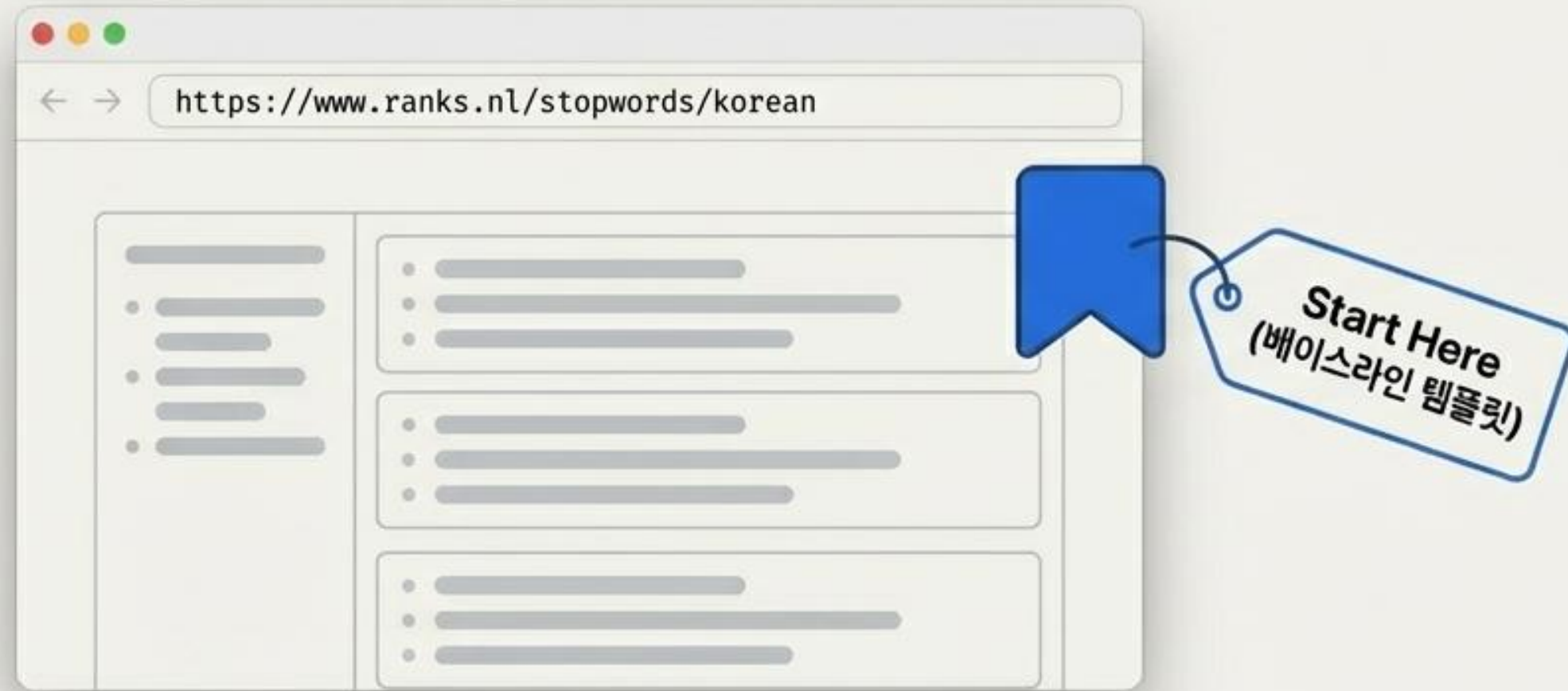


외부 파일 분리 및 런타임 Data Loader 아키텍처 도입

- 실무 프로젝트에서는 불용어 개수가 수천 개 이상으로 증가
- 코드와 데이터를 완벽히 격리하여 확장성 및 관리 효율성 극대화

실무 모범 사례 (Best Practice)

2: 한국어 불용어 표준 레퍼런스



Step 1. 표준 도입

업계 보편적 통용 템플릿 로드



Step 2. 도메인 평가

프로젝트 특수성에 맞춘 단어 추가/삭제



Step 3. 지속 고도화

분석 모델 피드백 루프를 통한 커스텀 사전 최적화

